프젝 총 정리 최종ver

CAN 기반 분산 ECU 무인 배달 차량 시스템

프로젝트 전체 정리 v9.5

최종 업데이트: 2026년 3월 9일

> **v9.5 변경사항**: delivery_mqtt_merge10.cpp (RPi1) / delivery_server.cpp (RPi3) 실제 코드 기준으로 파라미터·상태머신·MQTT 토픽 전면 검증. MQTT 토픽 통일 및 PIN 전달 경로 완성 확인(DONE). O_NONBLOCK 미제거 버그 발견 및 기록.

---

1. 프로젝트 개요

한 줄 요약

> **[STM32 × 2 + Raspberry Pi × 3] 분산 ECU 아키텍처**로 OpenCV 라인트레이싱 기반 자율 주행, PIN 인증 화물함 제어, Qt 기반 주문 클라이언트, MQTT 실시간 관제를 구현하는 무인 배달 차량 시스템

목적

실제 차량의 ECU 분산 구조(주행 / 미션 / 화물함)를 RC카로 재현하여 CAN 통신 기반 임베디드 시스템 설계 역량을 검증한다.

비즈니스 가치

음식 배달, 캠퍼스 내 무인 배송, 물류 창고 자동화 등 반복적인 배달 업무를 자동화한다.
수신자 PIN 인증 기반 화물함 제어로 분실·오배달 사고를 원천 차단하여 배달 신뢰성을 확보한다.

핵심 기술 선택 근거

기술	선택 이유
CAN 통신 (250 kbps)	주행/미션/화물함 ECU 분리, 단일 버스로 확장 가능. 버스 점유율 1% 미만
STM32 주행 ECU (C)	모터 PWM / 엔코더 / PID는 마이크로초 정밀도 필요 — RPi로 대체 불가
RPi1 미션 ECU (C++)	OpenCV 라인트레이싱 + ArUco 마커 인식 + 복잡한 미션 로직 — MCU 처리 불가
STM103 화물함 ECU (C)	CAN + GPIO + I2C + PWM만 필요. bxCAN 내장으로 MCP2515 불필요. 인터럽트 기반 실시간 처리
MQTT	다수 차량 확장 시 브로커 연결만으로 통신 가능. 차량당 연결 1개(RPi1)로 관리 단순화
스키드 스티어링	서보 조향 불필요. L298N 1개로 좌우 바퀴를 탱크 방식으로 제어
ArUco 마커	카메라 1대로 라인트레이싱과 목적지 인식을 동시 처리. ID별 목적지 구분
PIN 인증	주문 시 4자리 설정, 수령 시 키패드 입력으로 수신자만 화물함 접근 가능

2. 전체 시스템 구조

2-1. 노드 구성

구분	보드	위치	역할
STM32	B-L475E-IOT01A (STM32L4)	RC카 탑재	주행 ECU — 모터/엔코더/PID
RPi1	Raspberry Pi	RC카 탑재	미션 ECU — 라인트레이싱/ArUco/미션 로직/MQTT 게이트웨이
STM103	STM32F103 독립 보드	RC카 탑재	화물함 ECU — PIN 인증/서보/LCD/키패드
RPi3	Raspberry Pi	오프보드 (서버)	MQTT 브로커 / 관제 / DB
RPi4	Raspberry Pi	오프보드 (클라이언트)	Qt 주문 클라이언트 / 5인치 터치 디스플레이
서브 STM103	STM32F103 독립 보드	개발용	RPi1 대용 CAN 테스트 시뮬레이터

2-2. 통신 구조

```
┌──────────────────────────────────────────────┐
│                RC카 (온보드)                  │
│                                              │
│  STM32 (B-L475E) ──────┐                    │
│  (주행 ECU)            │                    │
│                       CAN 버스 (250 kbps)   │
│  RPi1 ────────────────┤                    │
│  (미션 ECU)            │   Wi-Fi/MQTT       │
│                       │   └──────────────── ├──→ RPi3 (서버)
│  STM103 ──────────────┘                    │         │
│  (화물함 ECU)                               │         │ MQTT
└──────────────────────────────────────────────┘     RPi4 (클라이언트)

온보드 CAN:    STM32 ↔ RPi1 ↔ STM103  ← 단일 CAN 버스 250 kbps
오프보드 MQTT: RPi1  ↔ RPi3           ← 무선 (차량당 단일 연결)
               RPi3  ↔ RPi4           ← 무선
```

2-3. 각 노드의 역할 분담 원칙

- **STM32 주행 ECU**: "판단하지 않는다." 명령 수행 + 센서 데이터 수집만 담당
- **RPi1 미션 ECU**: "모든 판단은 여기서 한다." 서버와의 통신도 RPi1만 담당
- **STM103 화물함 ECU**: 주행 ECU와 완전 독립된 보안 영역. CAN만 사용. 외부 네트워크 직접 연결 없음
- **RPi3 서버**: "실시간 제어에 절대 관여하지 않는다." 이벤트 기반 데이터 중계 및 저장
- **RPi4 클라이언트**: 고객 주문 접수 및 PIN 설정

---

3. 전체 배달 시나리오

3-1. 전체 흐름 요약

```
[① 고객 주문]
  RPi4 (Qt UI) → 목적지 + PIN 설정
  RPi4 → MQTT → RPi3: 주문정보 + PIN
  RPi3: DB 저장
  RPi3 → MQTT → RPi1: destination + pin + order_id
  RPi1 → CAN 0x012 → STM103: PIN(4자리) + 목적지 코드 전달
  RPi1: S_WAIT_CMD → S_FOLLOW (출발)

[② 자율 주행]
  RPi1: 카메라 → 라인트레이싱 → CAN 0x010 → STM32
  STM32: PID 속도제어 → L298N → 모터
  교차로 감지 시: 목적지별 교차로 행동 테이블에 따라 좌/우/직진

[③ 목적지 도착]
  RPi1: ArUco 마커 ID(1~4) 감지 → 즉시 정지
  RPi1 → CAN 0x013 → STM103: 도착 신호
  RPi1 → MQTT → RPi3: arrived 보고
  STM103: STATE_VERIFY_DEST 진입

[④ 화물함 인증 및 수령]
  소비자: LCD 확인 → 목적지 키(A/B/C/D) 입력 → PIN 4자리 입력 (#제출)

  [성공]
    STM103: 서보 열림 (5초 카운트다운) → 서보 닫힘
    STM103 → CAN 0x302=0x01 → RPi1: 인증 성공
    STM103 → CAN 0x301=0x00 → RPi1: 도어 닫힘
    RPi1 → MQTT alert(pin_success) → RPi3

  [실패 5회]
    STM103 → CAN 0x303=0x01 → RPi1: 잠금
    RPi1 → MQTT alert(pin_locked) → RPi3 로그
    10초 잠금 후 PIN 재입력 허용

  [오배달 / 미수령 타임아웃]
    STM103 → CAN 0x302=0x02 → RPi1: 즉시 유턴

[⑤ 귀환]
  RPi1: g_door_closed = true → S_UTURN (U턴)
  → S_REACQUIRE_AFTER_UTURN (직진, 라인 재탐색)
  → S_FOLLOW (returning=true, 귀환 교차로 행동 테이블)
  → ArUco ID=0 (출발지) 감지 → S_FINISHED
  RPi1 → MQTT → RPi3: completed
```

3-2. 목적지별 교차로 행동 테이블

목적지	방향	교차로 1번	교차로 2번
A (dest=1)	GO	좌회전	—
B (dest=2)	GO	우회전	—
C (dest=3)	GO	직진	좌회전
D (dest=4)	GO	직진	우회전
A (dest=1)	RETURN	우회전	—
B (dest=2)	RETURN	좌회전	—
C (dest=3)	RETURN	우회전	직진
D (dest=4)	RETURN	좌회전	직진

---

4. CAN 프로토콜 (최종 확정)

4-1. 통신 속도

항목	값	근거
통신 속도	250 kbps	버스 점유율 1% 미만, 배선 노이즈 내성 유리
프레임 수	약 20~30개/초	50 ms 주기 메시지 기준

4-2. CAN ID 테이블

CAN ID	내용	송신	수신	주기
0x010	주행 명령 (dir + rpm)	RPi1	STM32	50 ms
0x011	E-Stop	RPi1	STM32	즉시
0x012	배달정보 (PIN + 목적지)	RPi1	STM103	이벤트
0x013	도착 신호	RPi1	STM103	이벤트
0x100	현재 속도 피드백 (RPM×100)	STM32	RPi1	50 ms
0x101	전방 거리 (VL53L0X, mm)	STM32	RPi1	50 ms
0x200	ECU Heartbeat (0xAA)	STM32	RPi1	100 ms
0x301	도어 상태 (0x00=닫힘)	STM103	RPi1	이벤트
0x302	인증 결과 (0x01=성공, 0x02=오배달/미수령)	STM103	RPi1	이벤트
0x303	PIN 5회 실패 잠금 (0x01)	STM103	RPi1	이벤트

> ★ **0x013 분리 배경**: 기존 0x010(주행 명령)과 도착 신호를 같은 ID로 사용하면 STM32가 도착 신호를 모터 명령으로 오인하여 오동작. 0x013으로 분리하여 STM103만 수신하도록 변경.

4-3. CAN 페이로드 상세

0x010 — 주행 명령 (RPi1 → STM32, DLC=3)

Byte	내용	값
[0]	방향	0=정지, 1=전진, 2=후진, 3=좌, 4=우, 5=U턴
[1]	RPM×10 상위 바이트	uint8

Byte	내용	값
[2]	RPM×10 하위 바이트	uint8

0x012 — 배달정보 (RPi1 → STM103, DLC=6)

Byte	내용	값
[0]	유효 패킷 식별	0x01
[1]	PIN[0] (숫자 0~9)	uint8
[2]	PIN[1]	uint8
[3]	PIN[2]	uint8
[4]	PIN[3]	uint8
[5]	목적지 코드	0x01=A, 0x02=B, 0x03=C, 0x04=D

0x013 — 도착 신호 (RPi1 → STM103, DLC=1)

Byte	내용	값
[0]	도착	0x01

0x301 — 도어 상태 (STM103 → RPi1, DLC=1)

Data[0]	의미
0x00	도어 닫힘 → RPi1 유턴 트리거
0x01	도어 열림

0x302 — 인증 결과 (STM103 → RPi1, DLC=1)

Data[0]	의미
0x01	PIN 인증 성공 → MQTT alert 송신
0x02	오배달 확인 / 미수령 타임아웃 → 즉시 유턴

0x303 — 잠금 (STM103 → RPi1, DLC=1)

Data[0]	의미
0x01	PIN 5회 실패 잠금 → MQTT alert 송신

0x100 — 속도 피드백 (STM32 → RPi1, DLC=8)

Byte	내용	타입
[0~1]	좌 RPM × 100	int16 Big-Endian
[2~3]	우 RPM × 100	int16 Big-Endian
[4~5]	좌 CCR (PWM 값)	uint16 Big-Endian
[6~7]	우 CCR (PWM 값)	uint16 Big-Endian

---

5. MQTT 토픽 구조

5-1. 전체 토픽 테이블 (실제 코드 기준 — v9.5 검증 완료)

Topic	방향	QoS	내용
`delivery/order/{order_id}/{destination}`	RPi4 → RPi3	1	주문 정보 (토픽에 목적지 포함, JSON에 pin·menus·order_id)
`delivery/order/{order_id}/3to4`	RPi3 → RPi4	1	주문 수신 ACK
`delivery/vehicle/{vehicle_id}/order`	RPi3 → RPi1	1	출동 명령 — **PIN 포함** (order_id, destination, pin, menus) ✅
`delivery/vehicle/{vehicle_id}/1to3`	RPi1 → RPi3	1	출발/도착/완료 보고 (status 필드로 구분)
`delivery/vehicle/{vehicle_id}/status`	RPi1 → RPi3	0	Heartbeat (2초 주기, status: idle/in_transit)
`delivery/vehicle/{vehicle_id}/alert`	RPi1 → RPi3	1	pin_locked 이벤트
`delivery/vehicle/{vehicle_id}/3to1`	RPi3 → RPi1	1	차량 정보 수신 ACK

> ✅ **토픽 불일치 해소 (v9.5 확인)**: 서버가 `delivery/vehicle/{id}/order`로 발행, RPi1(merge10)이 동일 토픽 구독 — 통일 완료.

> ✅ **PIN 전달 경로 완성 (v9.5 확인)**: RPi4→RPi3 주문에 PIN 포함 → 서버 DB 저장 → `send_order_to_vehicle()`이 PIN 포함 발행 → RPi1 `last_pin` 수신 → CAN 0x012로 STM103 전달. 전 구간 완성.

> ⚠️ **alert 토픽 미처리**: `delivery/vehicle/{id}/alert`가 서버에서 `handle_vehicle()`로 흘러들어가며 별도 분기 없음. RPi3에 alert 핸들러 추가 필요.

5-2. RPi3 → RPi1 출동 명령 페이로드

```
{
  "order_id": "Order_1",
  "vehicle_id": "vehicle_001",
  "destination": "A",
  "pin": "1234",
  "menus": "메뉴1, 메뉴2",
  "timestamp": "2026-03-09 10:00:00",
  "message_type": "order_assignment"
}
```

5-3. RPi4 → RPi3 주문 토픽 형식

```
토픽: delivery/order/{order_id}/{destination}
예시: delivery/order/Order_1/A

JSON 페이로드:
{
  "order_id": "Order_1",
  "pin": "1234",
```

```
  "menus": ["메뉴1", "메뉴2"],
  "vehicle_id": "vehicle_001"
}
```

> RPi3 서버는 토픽 경로에서 destination을 직접 파싱하고 JSON의 destination 필드를 덮어쓴다.

5-4. MQTT 브로커 접속 정보

```
브로커 IP: 10.42.0.1 (Pi5_MQTT_AP 핫스팟)
사용자명:  hoji
비밀번호:  1234
포트:      1883
```

와이파이 재연결 명령:

```
sudo nmcli device wifi rescan
sudo nmcli device wifi connect "Pi5_MQTT_AP" password "12345678"
# 또는 간단히
sudo nmcli device wifi connect "Pi5_MQTT_AP"
```

---

6. STM32 주행 ECU 상세 설계

6-1. 핀맵

기능	핀	CubeMX 설정	연결 대상
USART1_TX	PB6	USART1_TX	ST-LINK VCP (PC COM3)
USART1_RX	PB7	USART1_RX	ST-LINK VCP
CAN1_RX	PB8	CAN1_RX	SN65HVD230 RXD
CAN1_TX	PB9	CAN1_TX	SN65HVD230 TXD
TIM5_CH1	PA0	Encoder	왼쪽 엔코더 A (레벨시프터 경유)
TIM5_CH2	PA1	Encoder	왼쪽 엔코더 B (레벨시프터 경유)
TIM3_CH1	PA6	Encoder	오른쪽 엔코더 A (레벨시프터 경유)
TIM3_CH2	PA7	Encoder	오른쪽 엔코더 B (레벨시프터 경유)
TIM2_CH1	PA15	PWM	L298N ENA (왼쪽)
TIM2_CH3	PA2	PWM	L298N ENB (오른쪽)
GPIO_OUT	PC2	GPIO_Output	L298N IN1
GPIO_OUT	PC3	GPIO_Output	L298N IN2
GPIO_OUT	PC4	GPIO_Output	L298N IN3
GPIO_OUT	PC5	GPIO_Output	L298N IN4
I2C2_SCL	PB10	I2C2_SCL	VL53L0X (온보드)
I2C2_SDA	PB11	I2C2_SDA	VL53L0X (온보드)

6-2. CubeMX 설정

```
USART1: PB6(TX)/PB7(RX), 115200/8N1, NVIC Enable
CAN1:   PB8(RX)/PB9(TX), Prescaler=16, BS1=13TQ, BS2=2TQ → 250kbps
TIM2:   CH1(PA15)/CH3(PA2), Prescaler=79, Period=999 → 1kHz PWM
TIM5:   CH1(PA0)/CH2(PA1), Encoder Mode TI12, Period=65535
TIM3:   CH1(PA6)/CH2(PA7), Encoder Mode TI12, Period=65535 (우측)
TIM6:   Prescaler=79, Period=9999 → 10ms PID 인터럽트
TIM7:   HAL Timebase (SysTick 대체)
IWDG:   Prescaler=/32, Reload=300 → 300ms Watchdog
Clock:  HSI 16MHz → PLL → SYSCLK 80MHz
        SYS → Debug: Serial Wire (PA15 JTAG 해제 필수)
```

6-3. 핵심 파라미터

```
#define PWM_MAX             999
#define PWM_OFFSET          300.0f
#define PULSE_PER_REV       5280        // 실측값 확정
#define PID_INTERVAL_MS     10

#define TARGET_RPM_DEFAULT  100.0f
#define TARGET_RPM_MAX      350.0f

#define KP                  0.5f
#define KI                  0.05f       // 적분 게인 추가
#define KD                  0.0f
#define INTEGRAL_LIMIT      150.0f

#define PWM_FORWARD         400
#define PWM_TURN            900         // 회전 최대값
#define PWM_BACKWARD        320         // 후진 고정 PWM

#define CAN_CMD_TIMEOUT_MS  5000        // 5초간 CAN 미수신 시 자동 정지
```

6-4. 모터 제어 정책

방향	PID	동작
전진 (DIR_FORWARD=1)	ON	pid_enable=1, target_rpm 추종
후진 (DIR_BACKWARD=2)	OFF	PWM 320 고정
좌회전 (DIR_LEFT=3)	OFF	왼쪽 후진(500) / 오른쪽 전진(PWM_TURN)
우회전 (DIR_RIGHT=4)	OFF	왼쪽 전진(PWM_TURN) / 오른쪽 후진(500)
U턴 (DIR_UTURN=5)	OFF	왼쪽 후진(500) / 오른쪽 전진(500)
정지 (DIR_STOP=0)	OFF	PWM=0, 모든 핀 LOW, PID 리셋

```
전진: IN1=H IN2=L / IN3=H IN4=L
후진: IN1=L IN2=H / IN3=L IN4=H
좌:   IN1=L IN2=H(후진) / IN3=H IN4=L(전진)
우:   IN1=H IN2=L(전진) / IN3=L IN4=H(후진)
U턴: IN1=L IN2=H(후진) / IN3=H IN4=L(전진)
```

6-5. 타이밍 구조

```
TIM6 인터럽트 (10ms)
├── pid_flag = 1
└── can_timeout_cnt 증가

메인 루프
├── CAN RX 처리 (can_rx_flag)
├── CAN 타임아웃 체크 (5초 → Motor_Stop)
└── pid_flag 처리
    ├── Encoder_Update() → RPM 계산
    ├── PID_Compute() → PWM 출력 (전진 시만 활성)
    ├── CAN TX: 속도 피드백 (50ms, 0x100)
    └── CAN TX: Heartbeat (100ms, 0x200)
```

6-6. Fail-safe (IWDG Watchdog)

항목	값
타임아웃	300 ms
동작	RPi1 다운 시 MCU 리셋 → 모터 즉시 정지
CAN 소프트웨어 타임아웃	5000 ms — IWDG와 별도 추가

---

7. RPi1 미션 ECU 상세 설계

7-1. 하드웨어 연결

```
RPi1
├── MCP2515 (SPI) → CAN 버스
│    ├── 오실레이터: 8 MHz (실물 확인 완료)
│    ├── 전원: 5V (RPi 5V 핀 공급) — 3.3V 직결 금지
│    ├── 트랜시버: TJA1050 내장 (5V 전용)
│    └── J1 점퍼 제거 (중간 노드 — 내장 120Ω 비활성화)
├── Wi-Fi → MQTT → RPi3 (브로커: Pi5_MQTT_AP / 10.42.0.1)
└── USB 카메라 (/dev/video0) — 라인트레이싱 + ArUco 마커 목적지 인식
```

MCP2515 레벨시프터 적용 범위:

SPI 신호	방향	레벨시프터 필요	이유
MOSI	RPi(3.3V) → MCP2515	불필요	MCP2515가 3.3V 입력 허용
SCK	RPi(3.3V) → MCP2515	불필요	동일
CS	RPi(3.3V) → MCP2515	불필요	동일
MISO	MCP2515(5V) → RPi	**필수**	5V 출력이 RPi 3.3V 핀 손상 유발
INT	MCP2515(5V) → RPi	**필수**	동일

`/boot/config.txt` **설정:**

```
dtoverlay=mcp2515-can0,oscillator=8000000,interrupt=25
# 적용 후
sudo ip link set can0 up type can bitrate 250000
```

7-2. 상태머신 (delivery_mqtt_merge10.cpp 기준 — 실제 동작)

> ⚠️ **코드 검증 결과**: 아래는 **현재 코드(merge10) 실제 동작**이다. 이전 문서에 기록한 "라인감지 방식 전환" 항목들(g_reacq_dir, area 감지, REACQ_RPM)은 **아직 코드에 미반영**이며, 별도 계획 항목으로 분리한다.

```
S_WAIT_CMD
└→ MQTT 출동 명령 수신 (destination + pin + order_id)
   → CAN 0x012: PIN + 목적지 → STM103
   → last_junc = 0 (첫 교차로 쿨다운 없이 즉시 감지)
   → S_FOLLOW

S_FOLLOW  ← 라인트레이싱 + 교차로 감지 + ArUco 감지 (50ms 주기 CAN 0x010)
├→ 교차로 감지 (streak ≥ JUNC_STREAK=2) + junc_count < max_junc
|    → S_JUNC_STOP (DUR_JUNC_STOP_MS=800ms 정지)
|      → S_JUNC_LEFT  : DIR_LEFT  + TURN_RPM, DUR_TURN_MS=4000ms 고정 후
→ S_REACQUIRE
|      → S_JUNC_RIGHT : DIR_RIGHT + TURN_RPM, DUR_TURN_MS=4000ms 고정 후
→ S_REACQUIRE
|      → S_JUNC_STRAIGHT : DIR_FORWARD, DUR_STRAIGHT_MS=2000ms 고정 후 →
S_REACQUIRE
|
|    [S_REACQUIRE — 현재 코드]
|    - follow_line() 결과 방향으로 DEFAULT_RPM 이동
|    - DUR_REACQ_MS=500ms 경과 후 → S_FOLLOW
|    ※ 계획(미반영): g_reacq_dir 방향 유지, 5프레임 중앙확인, REACQ_RPM 저속
|
└→ 목적지 ArUco ID 감지 (returning=false)
     → 즉시 정지
     → CAN 0x013: 도착 신호 → STM103
     → MQTT: arrived 보고
     → S_DELIVER_WAIT
```

```
S_DELIVER_WAIT
├← CAN 0x301=0x00 수신 → g_door_closed = true → S_UTURN
└← CAN 0x302=0x02 수신 → g_door_closed = true → S_UTURN

S_UTURN — 현재 코드: DUR_UTURN_MS=12000ms 고정 후 → S_REACQUIRE_AFTER_UTUR
N
  ※ 계획(미반영): 최소 2000ms + area≥1500 라인감지 즉시 완료, 타임아웃 10000m
s
  S_REACQUIRE_AFTER_UTURN: DIR_FORWARD, DUR_REACQ_UTURN_MS=1000ms → S_F
OLLOW
  → last_junc = 0 (귀환 첫 교차로도 쿨다운 없이 즉시 인식)

S_FOLLOW (returning=true)
└→ ArUco ID=0 감지 → S_FINISHED

S_FINISHED
└→ MQTT: completed → S_WAIT_CMD
```

7-3. Vision 처리 파이프라인

```
USB 카메라 1대 (/dev/video0, 320×240)
│
├── [라인트레이싱] 하단 35% ROI
│    GaussianBlur → OTSU 이진화 → 컨투어 → 무게중심 → 오차(err) 계산
│    err > STRAIGHT_DEADBAND(50px) → LEFT / RIGHT
│    err ≤ STRAIGHT_DEADBAND      → FORWARD
│
├── [교차로 감지] 중간 22% ROI (H*0.40 ~ H*0.62)
│    이진화 → 좌/중/우 3분할 픽셀 비율 계산
│    L≥0.40 && M≥0.40 && R≥0.40  → J_PLUS (3방향)
│    L≥0.40 && R≥0.40            → J_T (T자)
│    한쪽 ≥ HIGH_TH_ONE(0.65)     → J_T (단방향 강화)
│    연속 streak ≥ JUNC_STREAK(2) → 교차로 확정
│
└── [ArUco 마커] DICT_4X4_50
     ID 0: 출발지 (귀환 완료 판단)
     ID 1~4: 목적지 A~D
```

7-4. 주행 파라미터 (delivery_mqtt_merge10.cpp 실제값)

```
// ── 속도 ──────────────────────────────────────────────────────────
#define DEFAULT_RPM           40.0f   // 라인트레이싱 기본 속도
#define TURN_RPM              150.0f  // 교차로 회전 / U턴 속도
// 미반영 계획: REACQ_RPM 30.0f (재탐색 저속), UTURN_RPM 200.0f (U턴 전용)
```

```
// ── 교차로 감지 ──────────────────────────────────────────────
#define STRAIGHT_DEADBAND     50      // 직진 판단 오차 허용 (px)
#define JUNC_STREAK           2       // 연속 감지 임계값
#define JUNC_START_DELAY      3000    // 출발 후 감지 시작 지연 (ms)
#define JUNC_COOLDOWN_MS      3500    // 교차로 처리 후 재감지 방지 (ms)
#define HIGH_TH               0.40f   // 3방향 교차로 픽셀 비율 임계값
#define HIGH_TH_ONE           0.65f   // 단방향 강화 임계값
#define MID_TH                0.40f   // 중앙 픽셀 비율 임계값

// ── 상태 지속 시간 ──────────────────────────────────────────
#define DUR_JUNC_STOP_MS      800     // 교차로 정지 (ms)
#define DUR_STRAIGHT_MS       2000    // 직진 통과 (ms)
#define DUR_TURN_MS           4000    // 좌/우회전 (ms) ← 고정값; 라인감지
방식 미반영
#define DUR_UTURN_MS          12000   // U턴 (ms) ← 고정값; 라인감지 방식
미반영
#define DUR_REACQ_MS          500     // 라인 재탐색 (ms) ← 1500ms 변경 계
획 미반영
#define DUR_REACQ_UTURN_MS    1000    // U턴 후 직진 재탐색 (ms)

// 미반영 계획: TURN_AREA_MIN 1500, TURN_AREA_MAX 8000 (라인감지 임계값)
// 미반영 계획: CENTER_STREAK 5, CENTER_DEADBAND 60px (재탐색 중앙 확인)
```

7-5. CAN 수신 처리

```
0x301 = 0x00 → g_door_closed = true → S_UTURN 트리거
0x302 = 0x01 → MQTT alert(pin_success) 전송
0x302 = 0x02 → g_door_closed = true → 즉시 S_UTURN (오배달/미수령)
0x303 = 0x01 → MQTT alert(pin_locked) 전송 → RPi3 서버 로그
```

7-6. 빌드 및 실행

```
g++ delivery_mqtt.cpp -o delivery \
    $(pkg-config --cflags --libs opencv4) \
    -lmosquittopp -lpthread

./delivery             # 기본 브로커 10.42.0.1
./delivery 10.42.0.1   # 브로커 IP 명시
```

---

8. STM103 화물함 ECU 상세 설계

8-1. 핀맵 (최종 확정)

기능	핀	CubeMX 설정	연결 대상
CAN1_RX	PB8	CAN1_RX	SN65HVD230 RXD (AFIO Remap2)
CAN1_TX	PB9	CAN1_TX	SN65HVD230 TXD (AFIO Remap2)
I2C2_SCL	PB10	I2C2_SCL	16×2 LCD (HD44780, I2C)
I2C2_SDA	PB11	I2C2_SDA	16×2 LCD
TIM3_CH1	PA6	PWM	서보 모터 (50Hz)
LED	PA5	GPIO_Output	상태 표시 LED
USART1_TX	PA9	USART1_TX	디버그 115200 bps
USART1_RX	PA10	USART1_RX	디버그
Keypad Row1~4	PC0~PC3	GPIO_Output	4×4 키패드 행
Keypad Col1~4	PB12~PB15	GPIO_Input Pull-Up	4×4 키패드 열

8-2. CubeMX 설정

```
CAN1:   PB8(RX)/PB9(TX), Prescaler=6, BS1=13TQ, BS2=2TQ → 250kbps
        AFIO Remap2 적용 필수: __HAL_AFIO_REMAP_CAN1_2()
I2C2:   PB10(SCL)/PB11(SDA), Standard Mode 100kHz
TIM3:   CH1(PA6), Prescaler=47, Period=19999 → 50Hz (서보 표준)
USART1: PA9(TX)/PA10(RX), 115200/8N1
GPIO:   PC0~PC3 Output / PB12~PB15 Input Pull-Up
Clock:  HSI 8MHz → PLL×12 → SYSCLK 48MHz
        APB1 /2 → PCLK1 24MHz (CAN/I2C2/TIM3)
```

8-3. 서보 PWM 매크로

```
#define SERVO_OPEN   2000   // 도어 열림 펄스폭 (μs)
#define SERVO_CLOSE  1000   // 도어 닫힘 펄스폭 (μs)
```

8-4. 키패드 레이아웃

```
        Col1(PB12) Col2(PB13) Col3(PB14) Col4(PB15)
Row1(PC0)   1          2          3          A
Row2(PC1)   4          5          6          B
Row3(PC2)   7          8          9          C
Row4(PC3)   *          0          #          D

* = 한 자리 삭제 (백스페이스)
# = 확인 (제출)
A~D = 목적지 입력
```

8-5. 보안 상태머신

```
STATE_IDLE
└→ CAN 0x012 수신 (PIN + 목적지)
   LCD: "Destination: X / PIN received!"
   → STATE_PKG_RECEIVED

STATE_PKG_RECEIVED
└→ CAN 0x013 수신 (도착 신호)
   LCD: "Destination? / Enter: A B C D"
   auth_timeout 카운트 시작
   → STATE_VERIFY_DEST

STATE_VERIFY_DEST
├→ 키패드 입력 (A/B/C/D) — 목적지 일치
│   LCD: "Pkg arrived!! / PW: ____"
│   → STATE_WAIT_FOR_PIN
└→ 키패드 입력 — 목적지 불일치
    LCD: "Is dest X right? / 1:YES  2:NO"
    → STATE_WRONG_DEST_CONFIRM

STATE_WRONG_DEST_CONFIRM
├→ '1'(YES) → CAN 0x302=0x02 → STATE_IDLE (오배달 귀환)
└→ '2'(NO)  → STATE_VERIFY_DEST (재입력)

STATE_WAIT_FOR_PIN
├→ PIN 성공 (#제출)
│   LED ON + 서보 열림 (5초 논블로킹 카운트다운)
│   CAN 0x302=0x01 (인증 성공)
│   CAN 0x301=0x00 (도어 닫힘 → RPi1 유턴 트리거)
│   → STATE_IDLE
├→ PIN 실패 (1~4회)
│   LED 점멸 + "Wrong PIN! / N tries left"
└→ PIN 실패 (5회)
    CAN 0x303=0x01 (잠금)
    → STATE_LOCKED (10초)

STATE_LOCKED
└→ 10초 후 → STATE_WAIT_FOR_PIN (fail_cnt 리셋)

미수령 타임아웃 (AUTH_TIMEOUT_MS=30000ms)
  VERIFY_DEST / WRONG_DEST_CONFIRM / WAIT_FOR_PIN 상태에서 30초 초과
  → CAN 0x302=0x02 → STATE_IDLE
```

8-6. LCD 표시 항목

상태	1행	2행
IDLE	" Cargo ECU "	" Waiting... "
PKG_RECEIVED	"Destination: X "	"PIN received! "
VERIFY_DEST	"Destination? "	"Enter: A B C D "
WRONG_DEST_CONFIRM	"Is dest X right?"	"1:YES 2:NO "
WAIT_FOR_PIN	"Pkg arrived!! "	"PW: **** "
성공	"Delivery Done! "	"Enjoy ur day! :D"
LOCKED	"!! LOCKED !! "	" Wait: Xs... "
오배달/미수령	"Wrong delivery! "	"Returning home "

---

9. RPi3 서버 상세 설계

9-1. 핵심 원칙

실시간 제어에 절대 관여하지 않는다. 이벤트 기반 데이터 중계 및 저장.

9-2. 기능 구성 (delivery_server.cpp 기준)

- Mosquitto MQTT 브로커 운용 (핫스팟 `Pi5_MQTT_AP`, IP `10.42.0.1`)
- RPi4 → RPi3 → RPi1 주문 중계 (**PIN 포함** ✅)
- 주문 정보 + Heartbeat SQLite DB 저장
- 토픽에서 destination 직접 파싱 (JSON 필드보다 토픽 우선)
- RPi4에 주문 수신 ACK 발송

9-3. DB 테이블 구조 (init_database() 기준)

```
-- 차량 정보 (RPi1 접속 기록)
vehicle_table(id, timestamp, vehicle_id)

-- 배송 주문 정보 (PIN 포함 ✅)
order_table(id, timestamp, order_id, destination, pin, menus, vehicle_i
d, status)

-- 차량 Heartbeat (2초 주기)
heartbeat_table(id, timestamp, vehicle_id, status)
```

> ✅ **PIN DB 저장 완성**: `order_table`에 `pin TEXT NOT NULL` 컬럼 포함. 저장 후 `verify_order_in_database()`로 PIN 일치 검증.

9-4. 구독 토픽 및 메시지 처리

```
// 구독
subscribe("delivery/order/+/+", 1);   // RPi4 주문 수신
subscribe("delivery/vehicle/+",  1);   // RPi1 차량/heartbeat 수신
```

```
// 메시지 분기
delivery/order/{id}/{dest}  → handle_order()
                              ├─ 토픽에서 destination 파싱 (A/B/C/D 1글자)
                              ├─ order_table DB 저장 (PIN 포함)
                              ├─ verify_order_in_database() 검증
                              ├─ send_order_to_vehicle() → RPi1에 PIN 포
함 전송
                              └─ delivery/order/{id}/3to4 → RPi4 ACK

delivery/vehicle/+          → heartbeat=true 이면 handle_heartbeat()
                              그 외                   handle_vehicle()
                              ※ alert 이벤트 별도 처리 없음 (⚠ 미구현)
```

9-5. 빌드 및 실행

```
g++ -std=c++17 delivery_server.cpp -o delivery_server \
    -lmosquitto -lmosquittopp -lsqlite3

./delivery_server [broker_ip]          # 기본: 10.42.0.1
MQTT_BROKER_HOST=10.42.0.1 ./delivery_server   # 환경변수로도 설정 가능

# DB 조회
sqlite3 /home/pi/catnip/database/delivery_system.db \
    "SELECT order_id, destination, pin, status FROM order_table;"
```

---

10. RPi4 클라이언트 상세 설계

Qt Widgets 기반 주문 UI (5인치 터치 디스플레이, XPT2046).
목적지(A~D) + PIN 4자리 설정 후 MQTT로 RPi3에 주문 전송.
배달 완료 알림 수신 및 표시.

---

11. 하드웨어 구성

11-1. CAN 버스 배선

> **직선(데이지체인) 버스 구조** 필수. 별형(Star) 배선 금지.
> **모든 노드의 GND를 반드시 공통 연결.**

```
[STM32 주행 ECU]          [RPi1 + MCP2515]          [STM103 화물함 ECU]
  (버스 끝단)               (중간 노드)                (버스 끝단)
  SN65HVD230                 TJA1050 내장               SN65HVD230
  종단저항 있음             J1 점퍼 제거               종단저항 있음
  220Ω || 220Ω = 110Ω      내장 120Ω 비활성화          220Ω || 220Ω = 110
Ω
```

```
        │                          │                          │
        └──────────── CANH ────────┴──────────── CANH ────────┘
        └──────────── CANL ────────┴──────────── CANL ────────┘
        └──────────── GND  ────────┴──────────── GND  ────────┘
```

SN65HVD230 연결 (STM32 / STM103 공통):

```
STM32/STM103 CAN_TX → SN65HVD230 TXD
SN65HVD230 RXD → STM32/STM103 CAN_RX
SN65HVD230 VCC = 3.3V  ← GND와 혼동 주의 (오결선으로 CAN 불통 경험)
SN65HVD230 GND = GND
```

11-2. STM32 실제 배선

```
STM32 → L298N:
  PA15 → ENA 직결 (3.3V PWM 직결, 레벨시프터 불필요)
  PA2  → ENB 직결
  PC2 → IN1 / PC3 → IN2 / PC4 → IN3 / PC5 → IN4

L298N → 모터:
  OUT1/OUT2 → 왼쪽 모터 2개 병렬
  OUT3/OUT4 → 오른쪽 모터 2개 병렬

엔코더 → 레벨시프터 → STM32:
  왼쪽 A/B → PA0/PA1 (TIM5)
  오른쪽 A/B → PA6/PA7 (TIM3)

전원:
  외부 12V → L298N VIN
  L298N 5V 출력 → 엔코더 VCC, 레벨시프터 HV
  STM32 3.3V → 레벨시프터 LV / 공통 GND 필수
```

11-3. 타이머 사용 현황 (STM32)

타이머	용도	핀
TIM2	PWM — ENA(CH1), ENB(CH3)	PA15, PA2
TIM3	Encoder Mode — 오른쪽	PA6, PA7
TIM5	Encoder Mode — 왼쪽	PA0, PA1
TIM6	PID 인터럽트 (10ms)	—
TIM7	HAL Timebase	—
IWDG	Watchdog 300ms	—

11-4. 부품 리스트

부품	수량	비고
JGB37-520 엔코더 모터	4개	좌우 대표 각 1개 사용
STM103 보드 (STM32F103)	2개	화물함 ECU 1개 + CAN 시뮬레이터 1개
RC카 섀시 키트	1개	
L298N 모터 드라이버	1개	
SN65HVD230 CAN 트랜시버	2개	STM32, STM103 각 1개
MCP2515 CAN 모듈 (5V)	2개	RPi1 전용 1개, 예비 1개
4채널 레벨시프터	2개	엔코더 4채널 + MCP2515 MISO/INT 2채널
220Ω 저항	4개	종단저항 (양 끝단 220Ω×2 병렬)
USB 카메라	1개	/dev/video0
소형 서보 모터	1개	화물함 도어
4×4 키패드	1개	
LCD 1602 (I2C, HD44780)	1개	
5인치 터치 디스플레이 (XPT2046)	1개	RPi4 클라이언트
배터리 + 홀더 (12V)	1세트	

12. Fail-safe 설계

상황	감지	대응
STM32 Heartbeat 누락	RPi1: 300ms 이상 0x200 미수신	E-Stop(0x011) + MQTT 서버 보고
RPi1 다운	STM32 IWDG 300ms 만료	MCU 리셋 → 모터 즉시 정지
CAN 명령 타임아웃	STM32: 5초간 0x010 미수신	Motor_Stop() 자동 실행
PIN 5회 실패	STM103 카운트	10초 잠금 + 0x303 → MQTT 로그
오배달	STM103 목적지 불일치 확인	0x302=0x02 → RPi1 즉시 유턴
미수령 타임아웃	STM103: 도착 후 30초 초과	0x302=0x02 → RPi1 즉시 유턴
MQTT 연결 끊김	RPi1 연결 상태 감지	관제 UI 경고 표시

13. 기술 스택

영역	기술
STM32 주행 ECU 펌웨어	C + STM32 HAL (CubeMX)
STM103 화물함 ECU 펌웨어	C + STM32 HAL (CubeMX)
서브 STM103 CAN 시뮬레이터	C + STM32 HAL (bxCAN)
라인트레이싱 / ArUco	C++ + OpenCV 4
미션 로직 (상태머신)	C++ + SocketCAN
MQTT 클라이언트	C++ + mosquittopp
MQTT 브로커	Mosquitto (Pi5_MQTT_AP)
주문 UI	C++ + Qt Widgets (RPi4, 5인치 터치)
관제 UI	C++ + Qt Widgets (RPi3)

영역	기술
서버 DB	SQLite (C++ API)
CAN 컨트롤러	MCP2515 (RPi1) / bxCAN 내장 (STM32, STM103)
CAN 트랜시버	SN65HVD230 (STM32, STM103) / TJA1050 내장 (MCP2515)
빌드	STM32CubeIDE / g++

---

14. 팀 구성 및 역할

담당	역할	담당 영역
구영모	FW-Drive + PM	STM32 주행 ECU (모터/엔코더/PID/CAN) + 전체 일정/문서
인수민	FW-Drive	STM32 주행 ECU 공동 담당
윤성진	Vision + System + 마일스톤	RPi1 카메라/라인트레이싱/ArUco/미션 상태머신/일정 관리
김민우	FW-Cargo	STM103 화물함 ECU (CAN/키패드/PIN/서보/LCD)
최지호	BE + Network	RPi3 서버 (Mosquitto/SQLite/MQTT/클라이언트 연동)

---

15. GitHub 브랜치 구조

```
main                          ← 최종 통합 (직접 push 금지)
dev                           ← 통합 테스트
├── feature/stm32
│    ├── feature/stm32-youngmo
│    └── feature/stm32-insumin
├── feature/stm103
├── feature/stm103-sim
├── feature/rpi1
│    ├── feature/rpi1-vision
│    └── feature/rpi1-system
├── feature/rpi3
└── feature/rpi4
```

커밋 컨벤션:

```
[STM32] CAN RX 0x010 주행 명령 수신 구현
[STM103] CAN 0x013 도착신호 처리 구현
[RPi1] 교차로 상태머신 유턴 로직 수정
[RPi3] alert 토픽 구독 및 로그 처리
[docs] CAN ID 0x013 추가 반영
```

---

16. CAN 통신 성공 판정 기준

Phase 1 — 내부 Loopback 테스트 (단일 노드)

```
hcan1.Init.Mode = CAN_MODE_LOOPBACK;
// TX → RX 콜백 100% 수신, DLC/ID/Data 일치
```

Phase 2 — 2노드 Normal 통신 (STM32 ↔ 서브 STM103)
ACK 포함 HAL_OK, TEC/REC 증가 없음, BUS-OFF 없음

Phase 3 — 3노드 통합 (STM32 + RPi1 + STM103)
0x010 주행 명령 50ms 주기 정상, 0x100 피드백 정상, 수신률 ≥ 99%

오류 상태 모니터링:

```
uint32_t esr = READ_REG(hcan1.Instance->ESR);
uint8_t tec  = (esr >> 16) & 0xFF;   // 송신 에러 카운터
uint8_t rec  = (esr >> 24) & 0xFF;   // 수신 에러 카운터
uint8_t lec  = (esr >> 4)  & 0x07;   // 마지막 에러 코드
// LEC=3(Ack Error): 수신 노드 없음 or 종단저항 미연결
```

---

17. 주요 트러블슈팅 이력

날짜	문제	원인	해결
03-03	UART 통신 안됨	ST-LINK VCP가 PB6/PB7에 내부 연결 (PA9/PA10 아님)	USART1 핀을 PB6(TX)/PB7(RX)로 변경
03-03	모터 동작 안됨	ENA/ENB 점퍼캡 미삽입 → L298N 출력 플로팅	점퍼캡 재삽입 확인
03-03	PWM 속도 제어 안됨	ENA/ENB 레벨시프터 신호 미도달	PA15/PA2 직결 (3.3V로 L298N 인식 확인)
03-05	RPM float 출력 안됨	newlib-nano float printf 기본 비활성화	링커 플래그 `-u _printf_float` 추가
03-05	엔코더 카운트 음수	A/B 채널 반대로 배선	배선 교체 (코드 수정 없음)
03-06	교차로 정지 후 미출발	S_JUNC_STOP → S_JUNC_STRAIGHT 전환 조건 누락	상태머신 전환 추가
03-06	유턴 미작동	도착 신호 CAN ID 0x010 충돌 → STM103 오수신	도착 신호 0x013으로 분리
03-07	DELIVER_WAIT 타임아웃 강제 유턴	도어 닫힘 전 타임아웃 발생	DELIVER_WAIT_MS 방식 유지, 도어닫힘(0x301=0x00)으로 유턴 트리거
03-08	STM103 CAN 불통	SN65HVD230 트랜시버 GND/VCC 오결선	배선 정정 (GND↔3.3V 핀 위치 확인)
03-08	STM103 플래시 실패	HardFault로 칩 잠금	STM32CubeProgrammer Under Reset + Full Erase → 재플래시
03-08	STM103 CAN Normal mode 진입 실패	PB8 pull-up 미적용, DBF 비트 미클리어	PB8 외부 pull-up 추가, 초기화 순서 정정
03-08	Pi5_MQTT_AP 재연결 안됨	RPi 재부팅 후 Wi-Fi 자동 연결 불가	`sudo nmcli device wifi connect "Pi5_MQTT_AP"`

날짜	문제	원인	해결
03-08	STM32 모터 미작동 (CAN 수신은 됨)	CAN 타임아웃 카운터 STOP 조건 충족	CAN_CMD_TIMEOUT_MS=5000, RPi1에서 50ms 주기 명령 전송 유지
03-09	교차로 인식 후 정지 안됨 / 직진 지속	`send_cmd()` 내부에서 매 호출마다 `fcntl(O_NONBLOCK)` 설정 → TX 버퍼 경합 시 STOP/LEFT 명령이 `EAGAIN` 으로 조용히 드롭됨	`send_cmd()` 에서 `fcntl(O_NONBLOCK)` 두 줄 제거, blocking write로 변경 ⚠️ **merge10 코드 line 190에 O_NONBLOCK 잔존 — 제거 필요**
03-09	Motor_Stop() 후 관성 지속 주행	`Motor_Stop()` IN1~4 모두 LOW → L298N 공회전 (coast) 모드 — 브레이크 미적용	IN1=IN2=HIGH, IN3=IN4=HIGH 로 변경 (L298N 브레이크 모드)
03-09	DIR_UTURN CAN 수신 시 차량 정지	`main.c` 에 `DIR_UTURN(5)` 정의 없음 → `switch` default(STOP)로 처리됨	`DIR_UTURN=5` 정의 + `Motor_UTurn()` 추가 + `switch case` 추가
03-09	전진 시 PID 미작동	`Motor_Forward()` 내 `pid_enable=1` 설정 누락 — 코드 전체에서 pid_enable이 한 번도 1이 되지 않음	`Motor_Forward()` 내 `pid_enable = 1` 추가
03-09	적분 오버슈트 (RPM 불안정)	`KI=0.3f` 과다	`KI=0.3f` → `KI=0.05f`
03-09	교차로 좌회전 미작동 (목적지 A, C)	O_NONBLOCK 버그로 LEFT 명령 드롭 + 보드롱(배터리팩) 접촉 불량으로 모터 마찰 과다	`send_cmd()` blocking 수정 + 보드롱 → 전지(AA 건전지팩)로 교체, 마찰 해소 후 정상 동작 확인
03-09	좌회전 후 S_REACQUIRE에서 우회전 명령	REACQUIRE 진입 시 `follow_line` 이 치우친 라인 잡아 반대 방향 명령 출력	`g_reacq_dir` 변수 추가, 진입 방향 기억 후 중앙 정렬될 때까지 동일 방향 유지
03-09	회전 완료 후 라인 재탐색 미작동	REACQUIRE에서 한 프레임만 중앙 잡혀도 즉시 S_FOLLOW 전환	정지/회전 교대(200ms 주기) + 연속 5프레임 ±60px 유지 시 전환
03-09	라인 재탐색 중 맵 밖으로 이탈	REACQUIRE에서 라인 없어도 DEFAULT_RPM으로 직진	라인 없으면 완전 정지, 있으면 REACQ_RPM(30) 저속 이동
03-09	회전 타이머 고정값으로 과/부족 회전	`DUR_TURN_MS` 고정값 사용 — 환경에 따라 과회전/부족 발생	워밍업 800ms 후 area 1500~8000 감지 시 즉시 전환, 타임아웃 3000ms

[RPi1 / 라인트레이싱·교차로 관련]

날짜	문제	원인	해결
03-06	VNC에서 `cv::imshow()` 창 미출력	VNC 세션에 `DISPLAY` 환경변수 미설정	`DISPLAY=:0 ./delivery` 로 실행
03-06	직선 주행 중 교차로 오감지	`HIGH_TH=0.25` 가 너무 낮아 직선 구간도 교차로 조건 충족	`HIGH_TH=0.25 → 0.55` 상향, T자 조건을 `L && R ≥ HIGH_TH` 로 단순화
03-06	허프 변환 방식 교차로 감지 실패	카메라 45도 기울어짐으로 라인 자체가 수평선으로 감지 (H값 40~70 상	허프 변환 방식 포기 → L/M/R 흰색 비율 방식으로 최종 확정

날짜	문제	원인	해결
		시)	
03-06	컨투어 면적 방식 교차로 감지 실패	ROI가 넓어 직선과 교차로의 면적 차이 없음 (17000~27000 동일)	컨투어 면적 방식 포기 → L/M/R 비율 방식으로 확정
03-07	`junc_count` 4 이후에도 streak 계속 증가	교차로 처리 블록에 `junc_count < max` 조건 누락 → 무한 감지	감지 블록에 `junc_count < get_max_junc(dest)` 조건 추가
03-07	교차로 회전 각도 부족, 라인 이탈	`DUR_TURN_MS=750ms` 너무 짧음 + `Motor_Left` PWM 저값으로 회전력 부족	`DUR_TURN_MS=750 → 1200ms` 증가, 느린 쪽 PWM 30% → 60% 상향
03-07	RPi에서 `DEFAULT_RPM` 낮춰도 속도 변화 없음	`Motor_Forward()` 에서 `Motor_SetPWM_LR(400, 400)` 하드코딩 + `pid_enable=0`	`pid_enable=1` 추가로 PID 활성화 (단, `target_rpm=250` 기본값 함께 조정 필요)
03-08	귀환 중 ArUco ID=0 감지해도 미정지	`S_FOLLOW` 에서 `returning=true` 시 ID=0 정지 로직 누락	`if (returning && aid == 0) { state = S_FINISHED; }` 추가

[STM32 / RPi 개발 환경 관련]

날짜	문제	원인	해결
03-04	CJMCU-230 모듈 사용 시 CAN 반복 수신	1회 송신에도 동일 프레임이 MCU에서 반복 출력 — ACK 실패/물리계층 불안정으로 재전송 반복 추정	CJMCU-230 → **Waveshare CAN 모듈 (VP230)** 으로 교체, 동일 배선에서 정상 동작 확인
03-04	`write: No buffer space available` 반복	CAN TX 큐 기본 크기(10개) 초과	`sudo ip link set can0 txqueuelen 1000`
03-05	직진 명령 후 1초 뒤 반복 정지	`Motor_Forward()` 호출마다 `PID_Reset()` → PWM 0 초기화 반복 + 타임아웃 300ms 짧음	같은 방향 재명령 시 PID 리셋 방지 (`current_dir` 체크), `CAN_CMD_TIMEOUT_MS` 증가
03-05	PID가 PWM을 0으로 만들어 좌우 교대 정지	`TARGET_RPM_MAX=120` 설정 과소 (실제 모터 250~330RPM), `KP=8/KI=10` 과대	`TARGET_RPM_MAX=350`, `TARGET_RPM_DEFAULT=250`, `KP=0.5`, `KI=0.3`
03-05	회전 시 한쪽 바퀴만 동작	느린 쪽 PWM(30% = 150)이 모터 데드존 미만	느린 쪽 PWM 30% → 60%로 상향
03-05	STM32CubeIDE `main.h: No such file or directory`	`main.c` 를 `Core/Src` 가 아닌 `Core/Startup` 에 붙여넣음	`Core/Src/main.c` 파일을 IDE에서 직접 열어 편집
03-04	RPi SD카드 손상 (SSH 불가, HDMI 검정)	비정상 종료로 파일시스템 손상	Raspberry Pi Imager로 OS 재설치 (hostname/SSH 설정 포함)
03-04	VNC `Failed to start session`	wayvnc PAM 인증 실패 (`pam_unix setuid failed`)	`/etc/wayvnc/config` 에서 `enable_pam=false` 설정 또는 tightvnc 5901 포트 사용
03-04	`Cannot find device "can0"`	`/boot/firmware/config.txt` 에 MCP2515 드라이버 설정 누락	`dtparam=spi=on` + `dtoverlay=mcp2515-can0,oscillator=8000000,interrupt=25` 추가 후 reboot

날짜	문제	원인	해결
03-04	CAN `ERROR-PASSIVE`, candump 수신 없음	STM32 ACK 실패 — CAN 트랜시버 미연결	CJMCU-230 트랜시버 CANH/CANL 배선 재연결

18. 시연 환경

18-1. 권장 트랙 환경

항목	내용
바닥재	**EVA 매트** 권장 — 타일/나무 바닥 대비 마찰력 확보, 미끄러짐 방지
라인 테이프	**검은 테이프** (나무 책상 포함 모든 밝은 배경에서 사용 가능)
이진화 방식	`THRESH_BINARY_INV` — 밝은 배경, 검은 선 감지 (OTSU 자동 임계값으로 조명 변화 대응)
배터리	보드롱(배터리팩) 대신 **AA 건전지팩** 권장 — 접촉 저항 낮고 마찰 해소에 유리

18-2. 흰 테이프 사용 시 주의사항

흰 테이프 + 어두운 배경 조합 사용 시 코드 1줄 수정 필요:

```
// delivery_mqtt.cpp — follow_line(), classify_junction() 공통
// 변경 전 (검은 라인)
cv::threshold(blur, bin, 0, 255, cv::THRESH_BINARY_INV | cv::THRESH_OTS
U);

// 변경 후 (흰 라인)
cv::threshold(blur, bin, 0, 255, cv::THRESH_BINARY | cv::THRESH_OTSU);
```

18-3. 조명 주의사항

- OTSU 자동 임계값 덕분에 조명 변화에 어느 정도 강인하지만, 직사광선/역광은 오감지 유발 가능
- 형광등 균일 조명 환경 권장
- `HIGH_TH` 임계값(0.40)은 조명 조건에 따라 0.35~0.55 범위에서 조정

19. 미확정 / 튜닝 필요 항목

항목	현황
send_cmd() O_NONBLOCK 제거	merge10 코드 line 190에 아직 잔존 — 즉시 제거 필요
DUR_TURN_MS (4000ms)	환경/속도에 따라 과/부족 회전 — 라인감지 방식으로 전환 계획
DUR_UTURN_MS (12000ms)	환경/속도에 따라 과/부족 회전 — 라인감지 방식으로 전환 계획
DUR_REACQ_MS (500ms)	1500ms로 증가 계획 미반영
area 임계값 (1500, 8000)	라인감지 방식 구현 시 교차로 크기/조명에 따라 조정 필요
center_streak (5프레임), ±60px	라인감지 방식 구현 시 환경별 조정 필요

항목	현황
PID 게인 (Ki, Kd)	KP=0.5, KI=0.05 현재 적용 — Kd 추후 실험
SERVO_OPEN / SERVO_CLOSE	현재 2000/1000 — 실제 서보에 맞게 튜닝

부록. 현재까지 진행 사항

A. 완료된 항목

담당	항목
구영모/인수민	STM32: UART, 모터 PWM, 엔코더 RPM, PID 속도 제어, CAN 0x010 수신/0x100 송신
구영모/인수민	U턴(DIR_UTURN=5) 추가, KI=0.05 적용, PULSE_PER_REV=5280 실측 확정
구영모/인수민	Motor_Stop() 브레이크 모드 수정 (IN=HIGH), pid_enable=1 Motor_Forward() 내 추가
윤성진	delivery_mqtt_merge10.cpp 상태머신 완성 (교차로 / 유턴 / 귀환)
윤성진	ArUco + 라인트레이싱 단일 카메라 통합
윤성진	CAN + MQTT 통합, 브로커 Pi5_MQTT_AP 연동 확인
윤성진	MQTT 토픽 `/order` 구독 통일 (merge10)
김민우	STM103 키패드 / LCD / 서보 기본 동작 확인
김민우	STM103 CAN 0x012/0x013 수신, 0x301/0x302/0x303 송신 구현 완료
김민우	인증 상태머신 (PIN 입력 / 목적지 검증 / 잠금 / 타임아웃) 완성
최지호	Mosquitto MQTT 브로커 구축, RPi1 ↔ RPi3 ↔ RPi4 통신 확인
최지호	PIN 전달 경로 완성 — RPi4→RPi3 DB저장→RPi1(last_pin)→CAN 0x012→STM103 ✅
최지호	토픽 `delivery/vehicle/{id}/order`로 통일, 서버·RPi1 일치 확인 ✅

B. 남은 항목

항목	내용
[긴급] O_NONBLOCK 제거	merge10 `send_cmd()` line 190의 `fcntl(O_NONBLOCK)` 제거 — STOP/방향 명령 드롭 방지
[중요] RPi3 alert 핸들러 추가	`delivery/vehicle/{id}/alert` 수신 시 별도 분기 없음 → `handle_vehicle()` 내 event 필드 확인 로직 추가
3노드 CAN 통합 테스트	STM32 ↔ RPi1 ↔ STM103 실차 CAN 3노드 동작 확인
전체 배달 시나리오 E2E 테스트	RPi4 주문 → 자율주행 → 도착 → PIN 인증 → 귀환 전 구간
Fail-safe 검증	Heartbeat 누락 E-Stop, CAN 타임아웃 정지, PIN 잠금
DUR_TURN/UTURN_MS 라인감지 방식 전환	고정값 → area 감지 기반으로 코드 수정 (계획만 존재)
area/center_streak 튜닝	회전 완료 임계값(1500, 8000)과 중앙 정렬(5프레임, ±60px) 환경별 조정
데모 영상 촬영	최종 데모

문서 버전: v9.5 | 최종 업데이트: 2026-03-09